

ARIN5204: Reinforcement Learning

1.3 Markov Decision Processes

Dr. Long Chen

longchen@ust.hk

Assistant Professor, Dept. of CSE

The Hong Kong University of Science and Technology (HKUST)

February 23, 2026



- ① Markov Processes
- ② Markov Reward Processes
- ③ Markov Decision Processes
- ④ Optimal Policies and Optimal Value Functions

Some slides are from: David Silver (DeepMind), Bolei Zhou (UCLA), Katerina Fragkiadaki (CMU), Diana Borsa (DeepMind).

- 1 Markov Processes
- 2 Markov Reward Processes
- 3 Markov Decision Processes
- 4 Optimal Policies and Optimal Value Functions

Markov Property

Definition: Markov Property

A state s_t is **Markov** if and only if:

$$p(s_{t+1}|s_t) = p(s_{t+1}|s_1, s_2, \dots, s_t)$$

- The state captures all relevant information from the history
- One **the state** is known, **the history** may be thrown away
- *i.e.*, The state is a **sufficient statistic** of the future

Markov Process & State Transition Matrix

A **Markov process** is a memoryless random process, *i.e.*, a sequence of random state $s_1, s_2, \dots$ with the Markov property

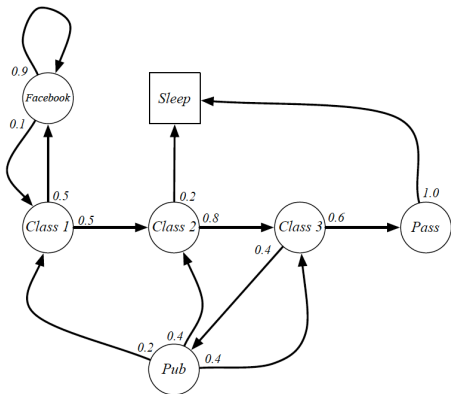
Definition: Markov Process

A **Markov Process (Markov Chain)** is a tuple $\langle \mathbf{S}, \mathbf{P} \rangle$:

- $\mathbf{S}$ is a (finite) set of states
- $\mathbf{P}$ is a state transition probability matrix, $\mathbf{P}(s'|s) = p(s_{t+1} = s' | s_t = s)$

$$\mathbf{P} = \begin{bmatrix} p(s_1|s_1) & p(s_2|s_1) & \cdots & p(s_N|s_1) \\ p(s_1|s_2) & & & \vdots \\ \vdots & & & \vdots \\ p(s_1|s_N) & \cdots & \cdots & p(s_N|s_N) \end{bmatrix}$$

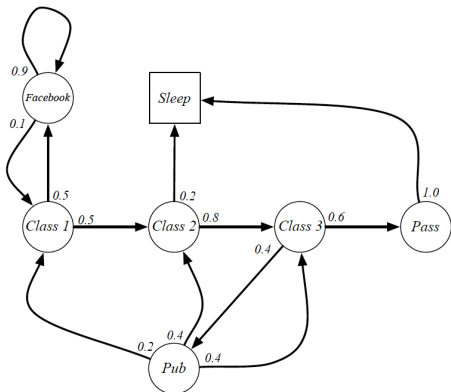
Example: Student Markov Chain



	C1	C2	C3	Pass	Pub	FB	Sleep
C1		0.5				0.5	
C2			0.8				0.2
C3				0.6	0.4		
Pass							1.0
Pub	0.2	0.4	0.4				
FB	0.1					0.9	
Sleep							1.0

Each row of the matrix sums to 1
 “Sleep” is the terminal state.

Example: Student Markov Chain



Sample **episodes** from Student Markov Chain starting from $s_1 = C_1$

- C1, C2, C3, Pass, Sleep
- C1 FB FB C1 C2 Sleep
- C1 C2 C3 Pub C2 C3 Pass Sleep
- C1 FB FB C1 C2 C3 Pub C1 FB
FB FB C1 C2 C3 Pub C2 Sleep

Episodic vs. Continuing Tasks

- **Episode tasks:** Interaction breaks naturally into episodes
 - e.g., plays of a game, trips through a maze
- **Continuing tasks:** Interaction does not have natural episodes, but just goes on and on..
 - Just like real life

- 1 Markov Processes
- 2 Markov Reward Processes
- 3 Markov Decision Processes
- 4 Optimal Policies and Optimal Value Functions

Markov Reward Process

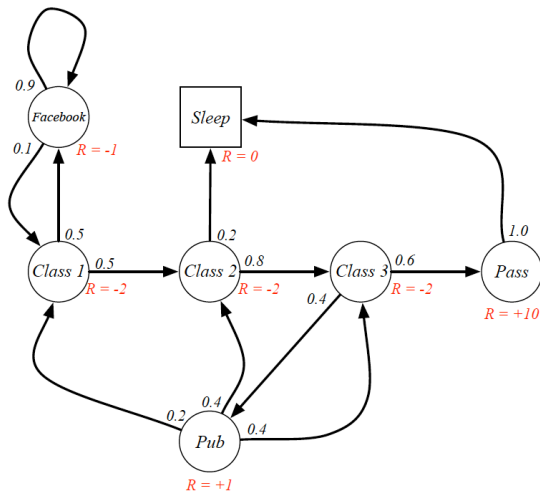
A **Markov reward process** is a Markov chain with values

Definition: Markov Reward Process

A **Markov Reward Process** is a tuple $\langle \mathbf{S}, \mathbf{P}, r, \gamma \rangle$:

- $\mathbf{S}$ is a (finite) set of states
- $\mathbf{P}$ is a state transition probability matrix, $\mathbf{P}(s'|s) = p(s_{t+1} = s' | s_t = s)$
- r is a reward function, $r_t = r(s_t = s) = \mathbb{E}[r_i | s_t = s]$
- γ is a discount factor, $\gamma \in [0, 1]$

Example: Student MRP



Horizon & Return

Horizon

- Number of maximum time steps in each episode
- Can be **infinite**, otherwise called a **finite Markov (reward) Process**

Definition: Return

The **return** G_t is the total discounted reward from time step t .

$$G_t = r_t + \gamma \cdot r_{t+1} + \dots = \sum_{k=0}^{\infty} \gamma^k \cdot r_{t+k}$$

- The discount $\gamma \in [0, 1]$ is the present value of future rewards
- The value of receive reward r after k time steps is $\gamma^k \cdot r$
- $\gamma \rightarrow 0$ leads to **myopic** evaluation, $\gamma \rightarrow 1$ leads to **far-sighted** evaluation

Why Discount Factor γ

Most of the Markov reward and decision processes are discounted. Why?

- Mathematically convenient to discount reward
- Avoid infinite returns in cyclic Markov processes
- Uncertainty about the future may not be fully represented
- It is sometimes possible to use undiscounted Markov reward processes (*i.e.*, $\gamma = 1$), *e.g.*, if all sequences terminate.

Value Function

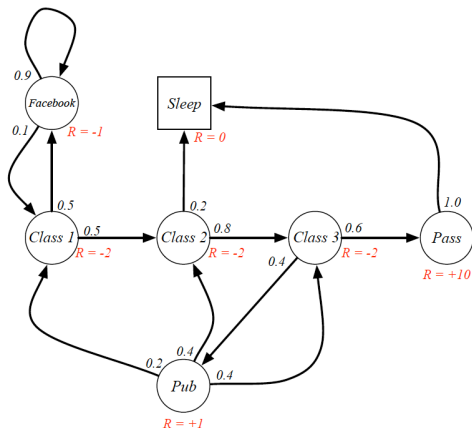
The value function $V(s)$ gives the long-term value of state s

Definition: Value Function

The **state value function** $V(s)$ of an MRP is the expected return starting from state s :

$$V(s) = \mathbb{E}[G_t | s_t = s]$$

Example: Student MRP Returns



Sample **return** from Student MRP starting from $s_1 = C_1$ with $\gamma = \frac{1}{2}$

- C1, C2, C3, Pass, Sleep

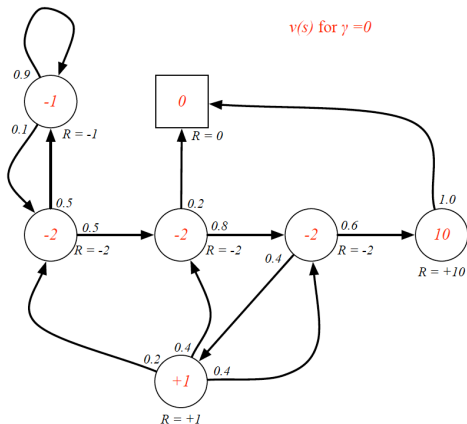
$$V_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 10 * \frac{1}{8} = -2.25$$
- C1 FB FB C1 C2 Sleep

$$V_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16} = -3.125$$
- C1 C2 C3 Pub C2 C3 Pass Sleep

$$V_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 1 * \frac{1}{8} - 2 * \frac{1}{16} \dots = -3.41$$

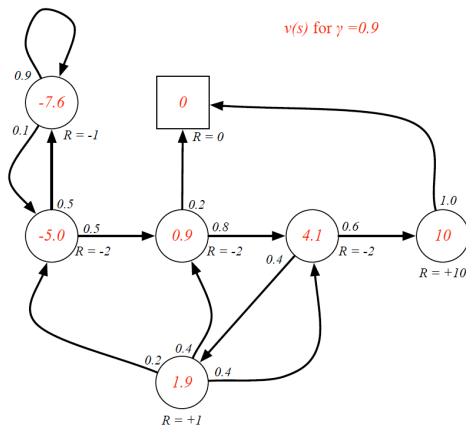
Example: State-Value Function for Student MRP

$V(s)$ for $\gamma = 0$



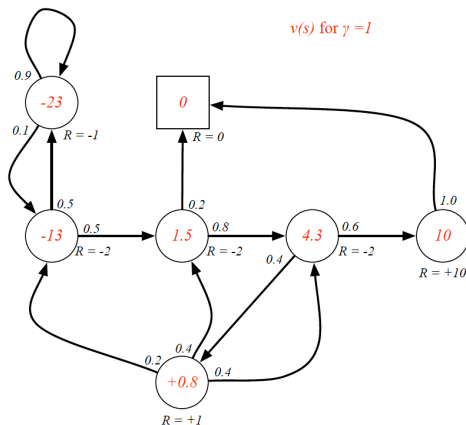
Example: State-Value Function for Student MRP

$V(s)$ for $\gamma = 0.9$



Example: State-Value Function for Student MRP

$V(s)$ for $\gamma = 1.0$



Bellman Equation for MRP

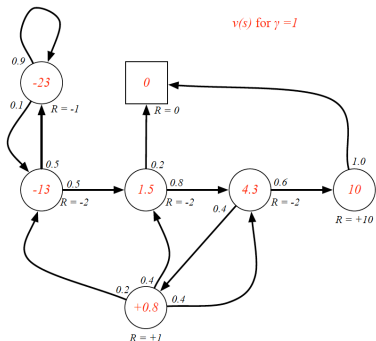
The value function can be decomposed into two parts:

- Immediate reward r_t
- Discounted value of successor state $\gamma V(s_{t+1})$

$$\begin{aligned} V(s_t) &= \mathbb{E}[G_t | s_t = s] \\ &= \mathbb{E}[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots | s_t = s] \\ &= \mathbb{E}[r_t + \gamma(r_{t+1} + \gamma r_{t+2} + \dots) | s_t = s] \\ &= \mathbb{E}[r_t + \gamma G_{t+1} | s_t = s] \\ &= \mathbb{E}[r_t + \gamma V(s_{t+1}) | s_t = s] \end{aligned}$$

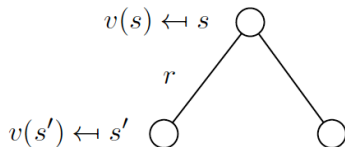
$V(s)$ for $\gamma = 1.0$

$$4.3 = -2 + 0.6 * 10 + 0.4 * 0.8$$



Bellman Equation for MRP

$$V(s) = \mathbb{E}[r(s_t) + \gamma V(s_{t+1}) | s_t = s]$$



$$V(s) = \text{immediate reward} + \sum_{s'} p(s'|s) \gamma \text{discounted sum of future reward}$$

immediate reward

discounted sum of future reward

Bellman Equation in Matrix Form

- The **Bellman equation** can be expressed concisely using matrices

$$\begin{bmatrix} V(s_1) \\ V(s_2) \\ \vdots \\ V(s_N) \end{bmatrix} = \begin{bmatrix} r(s_1) \\ r(s_2) \\ \vdots \\ r(s_N) \end{bmatrix} + \gamma \begin{bmatrix} p(s_1|s_1) & p(s_2|s_1) & \cdots & p(s_N|s_1) \\ p(s_1|s_2) & & & \vdots \\ \vdots & & & \vdots \\ p(s_1|s_N) & \cdots & \cdots & p(s_N|s_N) \end{bmatrix} \begin{bmatrix} V(s_1) \\ V(s_2) \\ \vdots \\ V(s_N) \end{bmatrix}$$

- The **Bellman equation** can be expressed concisely using matrices

$$\mathbf{V} = \mathbf{r} + \gamma \cdot \mathbf{P} \cdot \mathbf{V}$$

Solving the Bellman Equation

- The Bellman equation is a linear equation
- It can be solved directly:

$$\mathbf{V} = \mathbf{r} + \gamma \cdot \mathbf{P} \cdot \mathbf{V}$$

$$(\mathbf{I} - \gamma \mathbf{P}) \mathbf{V} = \mathbf{r}$$

$$\mathbf{V} = (\mathbf{I} - \gamma \mathbf{P})^{-1} \mathbf{r}$$

- Computational complexity is $O(N^3)$ for N states
- Direct solution only possible for small MRPs
- There are many iterative methods for large MRPs, e.g., dynamic programming, Monte-Carlo evaluation, temporal-difference learning

- 1 Markov Processes
- 2 Markov Reward Processes
- 3 Markov Decision Processes**
- 4 Optimal Policies and Optimal Value Functions

Markov Decision Process

A **Markov decision process** is a Markov reward process with decisions. It is an **environment** in which all states are Markov.

Definition: Markov Decision Process

A **Markov Decision Process** is a tuple $\langle \mathbf{S}, \mathbf{A}, \mathbf{P}, r, \gamma \rangle$:

- $\mathbf{S}$ is a (finite) set of states
- $\mathbf{A}$ is a (finite) set of actions
- $\mathbf{P}$ is a state transition probability matrix,

$$P(s'|s, a) = p(s_{t+1} = s' | s_t = s, a_t = a)$$
- r is a reward function, $r_t = r(s_t = s, a_t = a) = \mathbb{E}[r_i | s_t = s, a_t = a]$
- γ is a discount factor, $\gamma \in [0, 1]$

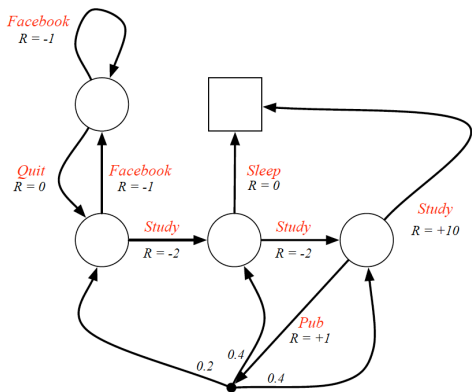
Markov Decision Process: Alternative Definition

Definition: Markov Decision Process

A **Markov Decision Process** is a tuple $\langle \mathbf{S}, \mathbf{A}, \mathbf{P}, \gamma \rangle$:

- $\mathbf{S}$ is a (finite) set of states
- $\mathbf{A}$ is a (finite) set of actions
- $\mathbf{P}$ is a joint probability of a reward r and next state s' , given a state s and action a , $\mathbf{P}(r, s' | s, a) = p(s_{t+1} = s', r_t = r | s_t = s, a_t = a)$
- γ is a discount factor, $\gamma \in [0, 1]$
- $\mathbf{P}$ defines the **dynamics** of the problem
- **There are equivalent formulations: no additional assumptions:**
 $p(s' | s, a) = \sum_r p(s', r | s, a)$ and $r(s, a) = \sum_r r \sum_{s'} p(s', r | s, a)$

Example: Student MDP



Policies in MDP

Definition: Policy

A **policy** π is a distribution over actions given states:

$$\pi(a|s) = p(a_t = a | s_t = s)$$

- A policy fully defines the behavior of an agent
- Policy specifies what action to take in each state
- MDP policies depend on the current state (not the history)
- Policies are stationary (time-independent), $a_t \sim \pi(a|s)$ for any $t > 0$.
- **Special case: deterministic policies**
 $\pi(s) = \text{the action taken with prob} = 1, \text{ when } s_t = s$

Policies in MDP

- Given an MDP $\langle \mathbf{S}, \mathbf{A}, \mathbf{P}, r, \gamma \rangle$ and a policy π
- **MDP to MP**: The state sequence $s_1, s_2, \dots$ is a Markov process $(\mathbf{S}, \mathbf{P}_\pi)$
- **MDP to MRP**: The state and reward sequence $s_1, r_1, s_2, r_2, \dots$ is a Markov reward process $\mathbf{S}, \mathbf{P}_\pi, r_\pi, \gamma$, where

$$p_\pi(s'|s) = \sum_{a \in \mathbf{A}} \pi(a|s) p(s'|s, a)$$

$$r_\pi(s) = \sum_{a \in \mathbf{A}} \pi(a|s) r(s, a)$$

Two Value Functions for MDP

Definition: State-Value Function

The **state-value function** $V_\pi(s)$ of an MDP is the expected return starting from state s , and **following policy π** : $V_\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$

Definition: Action-Value Function

The **action-value function** $Q_\pi(s, a)$ of an MDP is the expected return starting from state s with action a , and **following policy π** : $Q_\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$

- Value functions **capture the knowledge** of the agent regarding how good is each state for the goal she is trying to achieve (“...knowledge is represented as a large number of approximate value functions learned in parallel”)

Horde: A Scalable Real-time Architecture for Learning Knowledge from Unsupervised Sensorimotor Interaction. In AAMAS, 2011.

Two Value Functions for MDP

- The **state-value function** $V_\pi(s)$ of an MDP is the expected return starting from state s , and following policy π :

$$V_\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$$

- The **action-value function** $Q_\pi(s, a)$ of an MDP is the expected return starting from state s , taking action a , and following policy π :

$$Q_\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$$

- Relation with $V_\pi(s)$ and $Q_\pi(s, a)$:

$$V_\pi(s) = \sum_{a \in A} \pi(a|s) Q_\pi(s, a)$$

$$Q_\pi(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V_\pi(s')$$

Bellman Expectation Equation

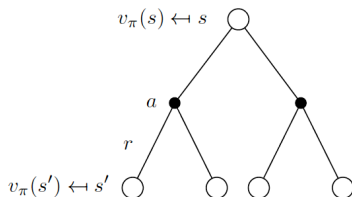
- The **state-value function** can be decomposed into **immediate reward** plus **discounted value of the successor state**

$$\begin{aligned} V_{\pi}(s) &= \mathbb{E}_{\pi}[G_t | s_t = s] \\ &= \mathbb{E}_{\pi}[r_t + \gamma V_{\pi}(s_{t+1}) | s_t = s] \end{aligned}$$

- The **action-value function** can be similarly decomposed

$$\begin{aligned} Q_{\pi}(s, a) &= \mathbb{E}_{\pi}[G_t | s_t = s, a_t = a] \\ &= \mathbb{E}_{\pi}[r_t + \gamma Q_{\pi}(s_{t+1}, a_{t+1}) | s_t = s, a_t = a] \end{aligned}$$

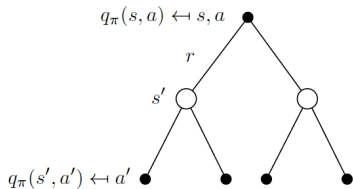
Bellman Expectation Equation (Backup Diagram) for V_π



$$V_\pi(s) = \sum_{a \in A} \pi(a|s) Q_\pi(s, a)$$

$$V_\pi(s) = \sum_{a \in A} \pi(a|s) \left(r(s, a) + \gamma \sum_{s' \in S} p(s'|s, a) V_\pi(s') \right)$$

Bellman Expectation Equation (Backup Diagram) for Q_π



$$Q_\pi(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V_\pi(s')$$

$$Q_\pi(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) \sum_{a' \in \mathcal{A}} \pi(a'|s') Q_\pi(s', a')$$

Summary: Bellman Expectation Equation for V_π and Q_π

$$V_\pi(s) = \sum_{a \in A} \pi(a|s) Q_\pi(s, a)$$

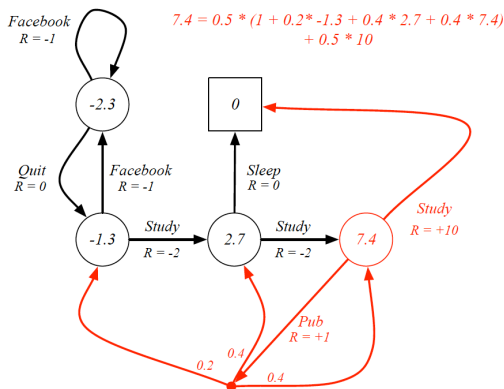
$$Q_\pi(s, a) = r(s, a) + \gamma \sum_{s' \in S} p(s'|s, a) V_\pi(s')$$

Thus,

$$V_\pi(s) = \sum_{a \in A} \pi(a|s) \left(r(s, a) + \gamma \sum_{s' \in S} p(s'|s, a) V_\pi(s') \right)$$

$$Q_\pi(s, a) = r(s, a) + \gamma \sum_{s' \in S} p(s'|s, a) \sum_{a' \in A} \pi(a'|s') Q_\pi(s', a')$$

Example: Bellman Expectation Equation in Student MDP



Two actions with 0.5 probabilities

$$7.4 = 0.5 * (1 + 0.2 * -1.3 + 0.4 * 2.7 + 0.4 * 7.4) + 0.5 * 10$$

Bellman Expected Equation (Matrix Form)

The Bellman expected equation can also be expressed concisely using the induced MRP:

$$V_{\pi} = \mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} V_{\pi}$$

with direct solution

$$V_{\pi} = (I - \gamma \mathbf{P}_{\pi})^{-1} \mathbf{r}_{\pi}$$

- 1 Markov Processes
- 2 Markov Reward Processes
- 3 Markov Decision Processes
- 4 Optimal Policies and Optimal Value Functions**

Optimal Value Function

Optimal State-Value & Action-Value Functions

- **Optimal state-value function $V_*(s)$** is the maximum value function over all policies

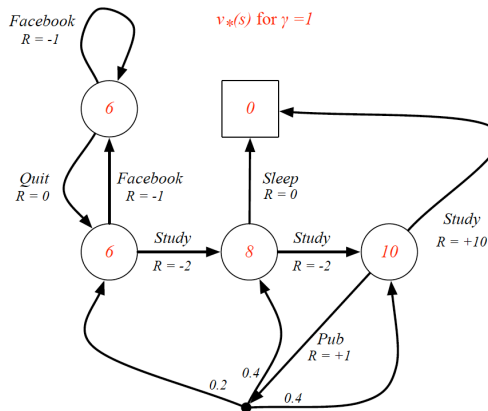
$$V_*(s) = \max_{\pi} V_{\pi}(s)$$

- **Optimal action-value function $Q_*(s, a)$** is the maximum action-value function over all policies

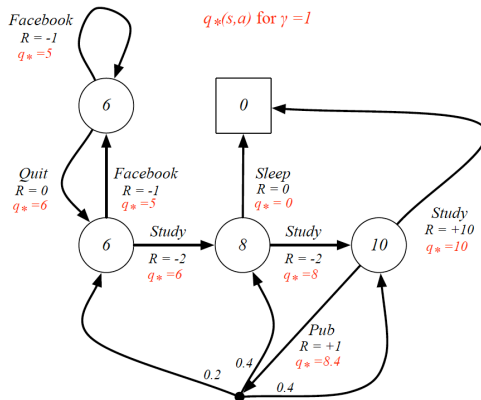
$$Q_*(s, a) = \max_{\pi} Q_{\pi}(s, a)$$

- Optimal value function specifies the best possible performance in the MDP
- An MDP is solved when we know the optimal value function

Example: Optimal State-Value Function for Student MDP



Example: Optimal Action-Value Function for Student MDP



Optimal Policy

Define a **partial ordering** over policies

$$\pi \geq \pi', \quad \text{if} \quad V_{\pi}(s) \geq V_{\pi'}(s), \forall s$$

Theorem

For any Markov Decision Process

- There exists an **optimal policy** π_* that is better than or equal to all other policies, $\pi_* \geq \pi, \forall \pi$
- All optimal policies achieve the optimal value function $V_{\pi_*}(s) = V_*(s)$
- All optimal policies achieve the optimal action-value function, $Q_{\pi_*}(s, a) = Q_*(s, a)$

Finding an Optimal Policy

- An optimal policy can be found by maximizing over $Q_*(s, a)$,

$$\pi_*(a|s) = \begin{cases} 1, & \text{if } a = \arg\max_{a \in A} Q_*(s, a) \\ 0, & \text{otherwise} \end{cases} \quad (1.1)$$

- There is always a deterministic optimal policy for any MDP
- If we know $Q_*(s, a)$, we immediately have the optimal policy, **we don't need the dynamics!**
- There can be **multiple optimal policies**
- If multiple actions maximize $Q_*(s, \cdot)$, we can also just pick any of these (including stochastically)

Finding an Optimal Policy

- An optimal policy can be found from the model dynamics using one step look ahead

$$\pi_*(a|s) = \begin{cases} 1, & \text{if } a = \arg\max_{a \in A} \sum_{s', r} p(s', r|s, a)(r + \gamma V_*(s')) \\ 0, & \text{otherwise} \end{cases}$$

- If we know $V_*(s)$, **we need the dynamics to do one step lookahead**, to choose the optimal action.

Policy Search

- One option is to enumerate search for the best policy
- Number of deterministic policies is $|A|^{|S|}$
- Other approaches, such as [policy iteration](#) and [value iteration](#), are more efficient (introduce later)

Policy Evaluation and Control in RL

Problem 1:

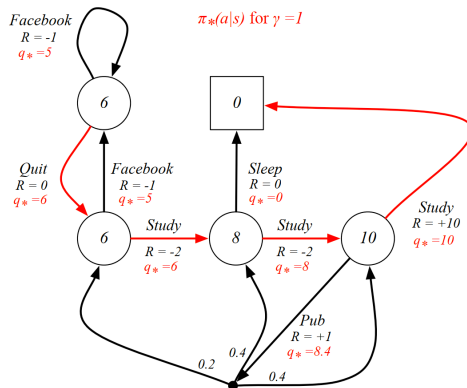
- Estimating V_π or Q_π is called **policy evaluation** or, simply, **prediction**
 - Given a policy, what is my expected return under that behaviour?

Problem 2:

- Estimating V_* or Q_* is sometimes called **control**, because these can be used for **policy optimization**
 - What is the optimal way of behaving? What is the optimal value function?

	State Values	Action Values
Prediction	V_π	Q_π
Control	V_*	Q_*

Example: Optimal Policy for Student MDP



Recap: Bellman Expectation Equations

Theorem

Given an MDP, $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma \rangle$, **for any policy π** , their value functions obey the following **expectation equations**:

$$V_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V_{\pi}(s') \right)$$

$$Q_{\pi}(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) \sum_{a' \in \mathcal{A}} \pi(a'|s') Q_{\pi}(s', a')$$

Bellman Optimality Equations

Theorem

Given an MDP, $\langle \mathbf{S}, \mathbf{A}, \mathbf{P}, r, \gamma \rangle$, the **optimal value functions** obey the following **optimality equations**:

$$V_*(s) = \max_a \left[r(s, a) + \gamma \sum_{s' \in \mathbf{S}} p(s'|s, a) V_*(s') \right]$$

$$Q_*(s, a) = r(s, a) + \gamma \sum_{s' \in \mathbf{S}} p(s'|s, a) \max_{a' \in \mathbf{A}} Q_*(s', a')$$

Some Intuition

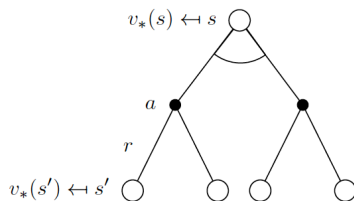
- An optimal policy can be found by maximizing over $Q_*(s, a)$,

$$\pi_*(a|s) = \begin{cases} 1, & \text{if } a = \arg\max_{a \in A} Q_*(s, a) \\ 0, & \text{otherwise} \end{cases} \quad (1.2)$$

- Apply the Bellman Expectation

$$\begin{aligned} Q_*(s, a) &= r(s, a) + \gamma \sum_{s' \in S} p(s'|s, a) \sum_{a' \in A} \pi_*(a'|s') Q_*(s', a') \\ &= r(s, a) + \gamma \sum_{s' \in S} p(s'|s, a) \max_{a' \in A} Q_*(s', a') \end{aligned}$$

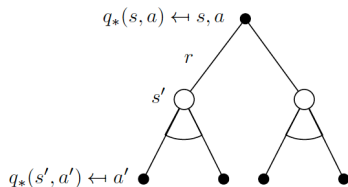
Bellman Optimality Equation for V_*



$$V_*(s) = \max_a Q_*(s, a)$$

$$V_*(s) = \max_a \left(r(s, a) + \gamma \sum_{s' \in S} p(s'|s, a) V_*(s') \right)$$

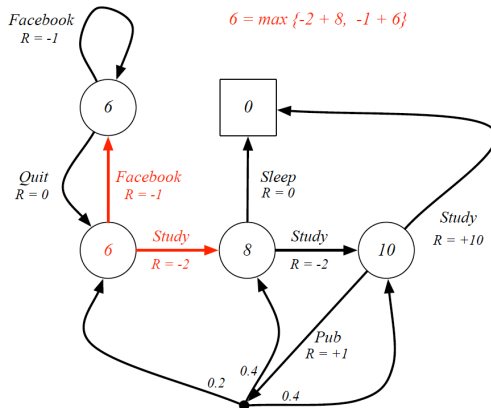
Bellman Optimality Equation for Q_*



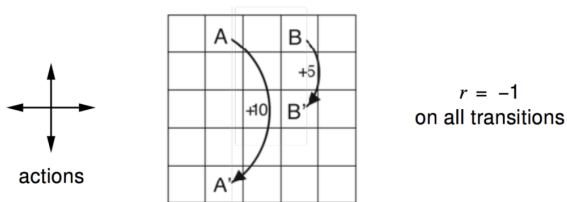
$$Q_*(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V_*(s)$$

$$Q_*(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) \max_{a'} Q_*(s', a')$$

Example: Bellman Optimality Equation in Student MDP



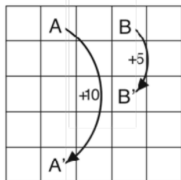
Another Example: Gridworld (S. & B. Ch3, Page 60)



- **Actions:** north, south, east, west
- If an action would take agent off the grid: no move but reward -1
- Other actions produce reward 0, except actions that move agent out of special state A and B as shown

Another Example: Gridworld (S. & B. Ch3, Page 60)

Equiprobable random policy ($\gamma = 0.9$)



(a)



Actions

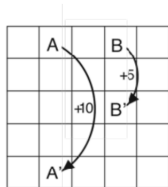
3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

(b)

$$4.43 = 0.25 * (0 + 0.9 * 5.3 + 0 + 0.9 * 2.3 + 0 + 0.9 * 8.8 + (-1) + 0.9 * 4.4)$$

Another Example: Gridworld (S. & B. Ch3, Page 60)

Optimal policy ($\gamma = 0.9$)



(a)



Actions

22.0	24.4	22.0	19.4	17.5
19.8	22.0	19.8	17.8	16.0
17.8	19.8	17.8	16.0	14.4
16.0	17.8	16.0	14.4	13.0
14.4	16.0	14.4	13.0	11.7

b) v_*

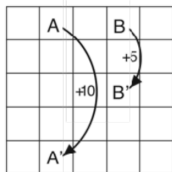
→	↕	←	↕	←
↖	↑	↖	←	←
↖	↑	↖	↖	↖
↖	↑	↖	↖	↖
↖	↑	↖	↖	↖

c) π_*

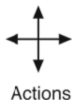
$$24.4 = 10 + 0.9 * 16.0$$

Another Example: Gridworld (S. & B. Ch3, Page 60)

Optimal policy ($\gamma = 0.9$)



(a)



22.0	24.4	22.0	19.4	17.5
19.8	22.0	19.8	17.8	16.0
17.8	19.8	17.8	16.0	14.4
16.0	17.8	16.0	14.4	13.0
14.4	16.0	14.4	13.0	11.7

b) v_*

→	↕	←	↕	←
↙	↑	↘	←	←
↙	↑	↘	↘	↘
↙	↑	↘	↘	↘
↙	↑	↘	↘	↘

c) π_*

$$22.0 = \max(0 + 0.9 * 19.4 + 0 + 0.9 * 19.8 + 0 + 0.9 * 24.4 + (-1) + 0.9 * 22)$$

Extra Reading Materials

- Chapter 3 (Finite Markov Decision Processes), Reinforcement Learning: An Introduction, Sutton & Barto.